
abstraction

Handling complexity by hiding unnecessary details.

Essentially, abstraction is a reminder to make sure that the user's view of a system is as simple as possible.

Abstraction is extremely common in consumer products. Many people use TVs, computers, microwave ovens, cars, and so on every day. We become comfortable with the user interface that these products extend to us. Through their controls, they provide an abstraction of their functionality. You press the accelerator pedal in a car to make it go forward. With a microwave, you set an amount of time and press Start to heat up some food. But few of us really know how these products work internally.

Here's an example of abstraction from the world of computer science. In programming, a *stack* is a mechanism for remembering data in a *last in, first out (LIFO)* order. Think of a pile of plates, where clean plates are added to the top and users take one from the top when they need a plate. A stack has two standard operations: `push` adds an item to the top of the stack, and `pop` removes the topmost item from the stack.

A stack is particularly useful whenever your program does any navigation because it can be used to leave a trail of breadcrumbs for finding your way back. This is how programming languages keep track of the execution of function and method calls in code: when you call a function or method, the return point is pushed onto a stack, and when the function or method returns, the place to return to is discovered by popping the most recent information off the top of the stack. In this way, code can make as many levels of calls as you need, and it always unwinds correctly.

As an abstraction, suppose a client program required the functionality of a stack that would be simple to create and would provide the ability to push and pop information. If this were written as a class, the client code would create a stack like this:

```
oStack = Stack()
```

A client would add information by calling a `push()` method like this: